본 저작물은 「저작권법」 제25조(학교교육 목적 등에의 이용)에 따라
적법하게 이용되고 있습니다.
이용 범위를 초과한 사용은 저작권법에 저촉될 수 있으므로,
저작물의 재복제 및 수업 목적 외 사용을 금지합니다.

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

아이디어를 현실로 - **바이브 코딩**

Git과 GitHub 버전 관리

버전 관리의 필요성

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

오늘의 학습 목표

1

버전 관리 이해하기

"최종_최최종" 문제 해결

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

오늘의 학습 목표

1. **버전 관리 이해하기**
   "최종_최최종" 문제 해결

2. **Git 핵심 용어 익히기**
   AI에게 요청할 때 필요한 5가지

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

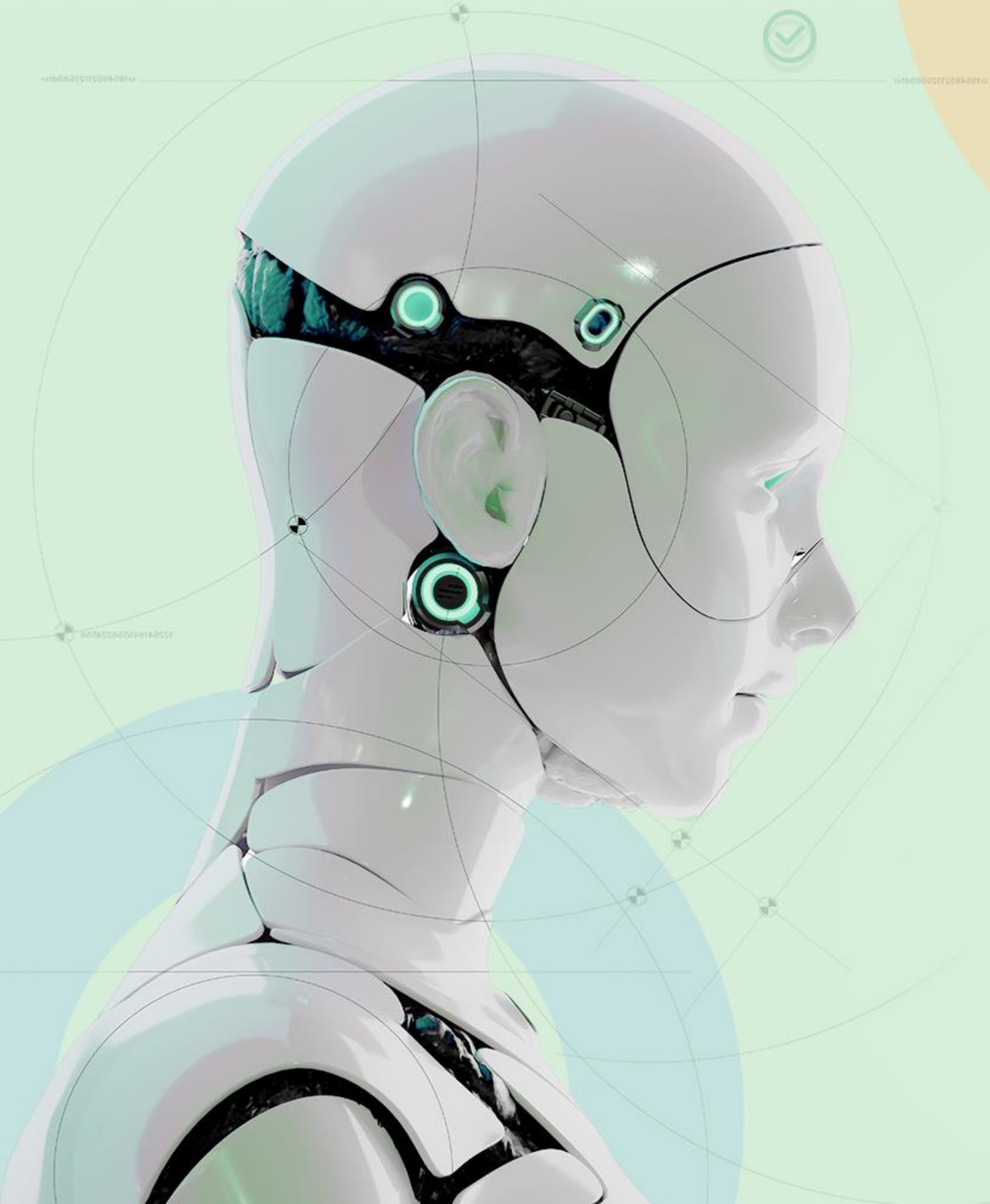
오늘의 학습 목표

1. **버전 관리 이해하기**
   "최종_최최종" 문제 해결

2. **Git 핵심 용어 익히기**
   AI에게 요청할 때 필요한 5가지

3. **GitHub 연동하기**
   프로젝트를 클라우드에 안전하게 보관

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.



CONTENTS

01 버전 관리의 필요성

02 Git 용어와 GitHub 준비

03 실습

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

마이링크 프로젝트 진행 상황

진행률 **10% → 15%**

1주차 ☑	2주차 ☑	3주차(현재)	4주차
환경 설정	기본 구조	Git 연동	스타일링

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

이 파일 이름, 익숙하신가요?

- 📄 기말레포트_최종.hwp
- 📄 기말레포트_최종_수정.hwp
- 📄 기말레포트_진짜최종.hwp
- 📊 팀프로젝트_발표_v3.pptx
- 📋 이력서_수정본_최신.pdf
- 📦 포트폴리오_제출용_이거임.zip

이런 경험, 다들 있으시죠? 😅

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

버전 관리가 없다면?

- ✕ 어떤 게 최신인지 모름
  파일명만으로는 판단 불가
- ✕ 이전 버전으로 돌아갈 수 없음
  실수했을 때 복구 불가

💡 코드는 문서보다 훨씬 자주, 훨씬 많이 수정됩니다!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

버전 관리가 없다면?

- ✕ **어떤 게 최신인지 모름**
  파일명만으로는 판단 불가
- ✕ **이전 버전으로 돌아갈 수 없음**
  실수했을 때 복구 불가
- ✕ **협업할 때 충돌**
  같은 파일을 여러 명이 수정하면 혼란
- ✕ **변경 이력 추적 불가**
  누가, 언제, 무엇을 바꿨는지 모름

💡 코드는 문서보다 훨씬 자주, 훨씬 많이 수정됩니다!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Git이란?

📦 코드의 타임머신

코드의 변경 이력을 자동으로 기록하는 도구

2005년 리눅스 토발즈가 개발

전 세계 개발자의 93% 이상이 사용

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Git의 특징

무료

완전 무료, 오픈소스

복구

이전 버전으로
되돌리기

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Git의 특징

무료
완전 무료, 오픈소스

복구
이전 버전으로 되돌리기

이력
변경 기록 자동 저장

협업
여러 명 동시 작업

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Git, 왜 배워야 할까?

개발자 면접 필수 질문
"GitHub 주소 있으세요?"

포트폴리오 역할
내가 뭘 만들었는지 증명

바이브 코딩으로 코드를 만들어도, 그 코드를 관리하는 건 Git!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Git, 왜 배워야 할까?

개발자 면접 필수 질문
"GitHub 주소 있으세요?"

포트폴리오 역할
내가 뭘 만들었는지 증명

협업의 기본
회사에서 팀 프로젝트 필수 도구

AI 시대에도 유효
AI가 코드를 만들어도 관리는 Git으로

바이브 코딩으로 코드를 만들어도, 그 코드를 관리하는 건 Git!

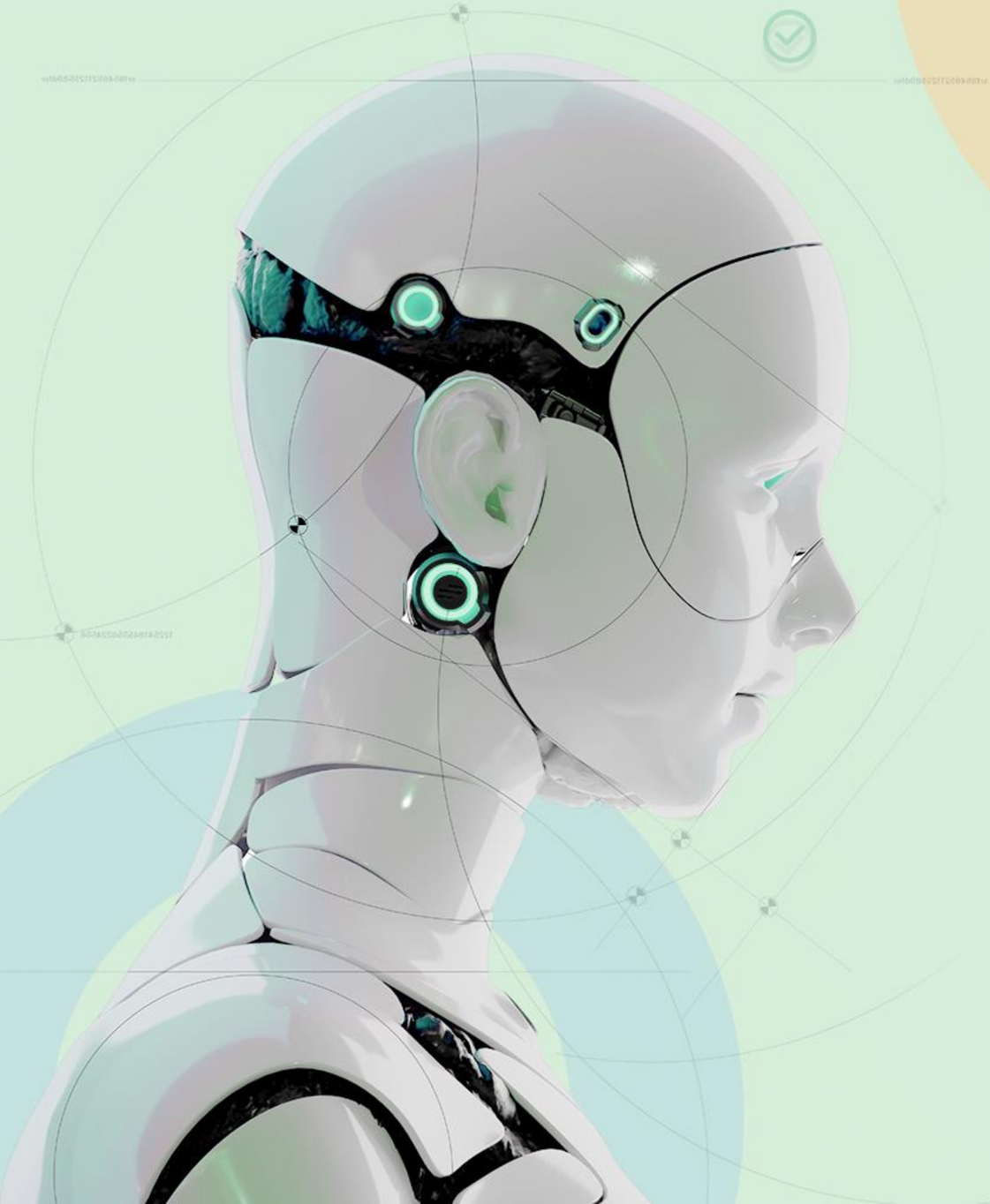
※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

아이디어를 현실로 - **바이브 코딩**

Git과 GitHub 버전 관리

Git 용어와 GitHub 준비

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.



CONTENTS

01 버전 관리의 필요성

02 Git 용어와 GitHub 준비

03 실습

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

지난 시간 복습

Git 용어와 GitHub 준비

📚 지난 차시에서 배운 것

- Git = 코드의 타임머신
- 버전 관리로 "최종_최최종" 해결

🎯 이번 차시 목표

- Git 핵심 용어 5개 익히기
- GitHub 가입 및 레포 생성
- AI로 연동 준비하기

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

바이브 코딩에서 알아야 할 Git 용어

용어	쉬운 설명	비유
Repository(레포)	프로젝트 저장소	프로젝트 폴더
Commit(커밋)	변경사항 저장	게임 세이브
Push(푸시)	서버에 업로드	클라우드 백업
Pull(풀)	서버에서 다운로드	동기화
Clone(클론)	레포 전체 복사	프로젝트 복사

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Git 작업 흐름

내 컴퓨터

commit → push
→
←
pull

GitHub

commit
내 컴퓨터에 저장
(세이브)

push
GitHub에 업로드
(백업)

pull
GitHub에서 다운로드
(동기화)

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

GitHub란?

🔧 **Git**

버전 관리 도구
(내 컴퓨터에 설치하는 프로그램)

≠

☁️ **GitHub**

Git 저장소를 온라인에
보관하는 서비스(웹사이트)

GitHub의 역할

- 코드를 클라우드에 안전하게 보관
- 다른 사람과 코드 공유
- 포트폴리오로 활용 가능

전 세계
1억 명+
이상 사용!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

레포지토리 만들기

1 → 2 → 3 → 4

+ 버튼 클릭
우측 상단 + 아이콘
→ "New repository"

이름 입력
Repository name에
"mylink" 입력

Public 선택
누구나 볼 수 있는
공개 저장소

Create 클릭
"Create repository"
버튼 클릭

☑ 결과: 빈 레포지토리 생성 완료!

주소가 생겨요: github.com/내아이디/mylink

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Git 설치하기

🪟 Windows

1. git-scm.com 접속
2. "Download for Windows" 클릭
3. 파일 실행 → Next만 계속 클릭!
4. 완료 후 명령 프롬프트에서 확인

🍎 Mac

1. 터미널 열기(Spotlight 검색)
2. git --version 입력
3. 설치 팝업 → "설치" 클릭
4. 10~15분 대기 후 완료

☑ 확인 방법

터미널/명령 프롬프트에서 git --version 입력
→ 버전 번호가 보이면 성공!(예: git version 2.40.0)

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

AI로 Git 연동하기

📋 준비물

1. GitHub 계정
   이미 가입되어 있다면 OK
2. 새로 만든 Repository 주소
   방금 만든 mylink 레포
3. 2주차 MyLink 프로젝트
   프로필 페이지 코드

🚀 전체 흐름

1. GitHub에서 빈 Repository 생성
2. "GitHub에 연동해줘" 요청
3. AI가 알아서 실행!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Gemini CLI 프롬프트 예시

1 GitHub 연동(처음 한 번만)

이 프로젝트를 GitHub에 연동해줘.
레포지토리 주소: https://github.com/내아이디/mylink

2 커밋(변경할 때마다)

현재 변경사항을 커밋해줘.

3 푸시(GitHub에 올릴 때)

GitHub에 푸시해줘.

💡 커밋 메시지는 AI가 변경 내용을 보고 알아서 작성해줘요!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

Gemini CLI 프롬프트 예시

2 커밋(변경할 때마다)

현재 변경사항을 커밋해줘.

3 푸시(GitHub에 올릴 때)

GitHub에 푸시해줘.

💡 \"커밋하고 푸시해줘\"로 한 번에 요청 가능!

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

연동 후 확인하기

🌐 GitHub 웹사이트에서 확인

1. github.com에 로그인
2. 내 Repository 클릭
3. **파일 목록이 보이면 성공! ✓**

📜 커밋 히스토리 확인

"Commits" 탭 클릭

→ 언제, 어떤 변경이 있었는지 확인 가능

🛠️ 문제가 생겼다면?

AI에게 "에러 메시지 보여줘" 또는 "다시 시도해줘" 요청

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

수정하고 다시 커밋하기

코드 수정 → 커밋 → 푸시 → (반복)

예시 시나리오

1. 프로필 이름 변경 → "커밋하고 푸시해줘" → AI: "이름 변경" 자동 생성
2. 소개글 추가 → "커밋하고 푸시해줘" → AI: "소개글 추가" 자동 생성
3. 이미지 추가 → "커밋하고 푸시해줘" → AI: "프로필 이미지 추가" 자동 생성

✓ 결과: 커밋 히스토리에 변경 이력이 자동으로 쌓임!

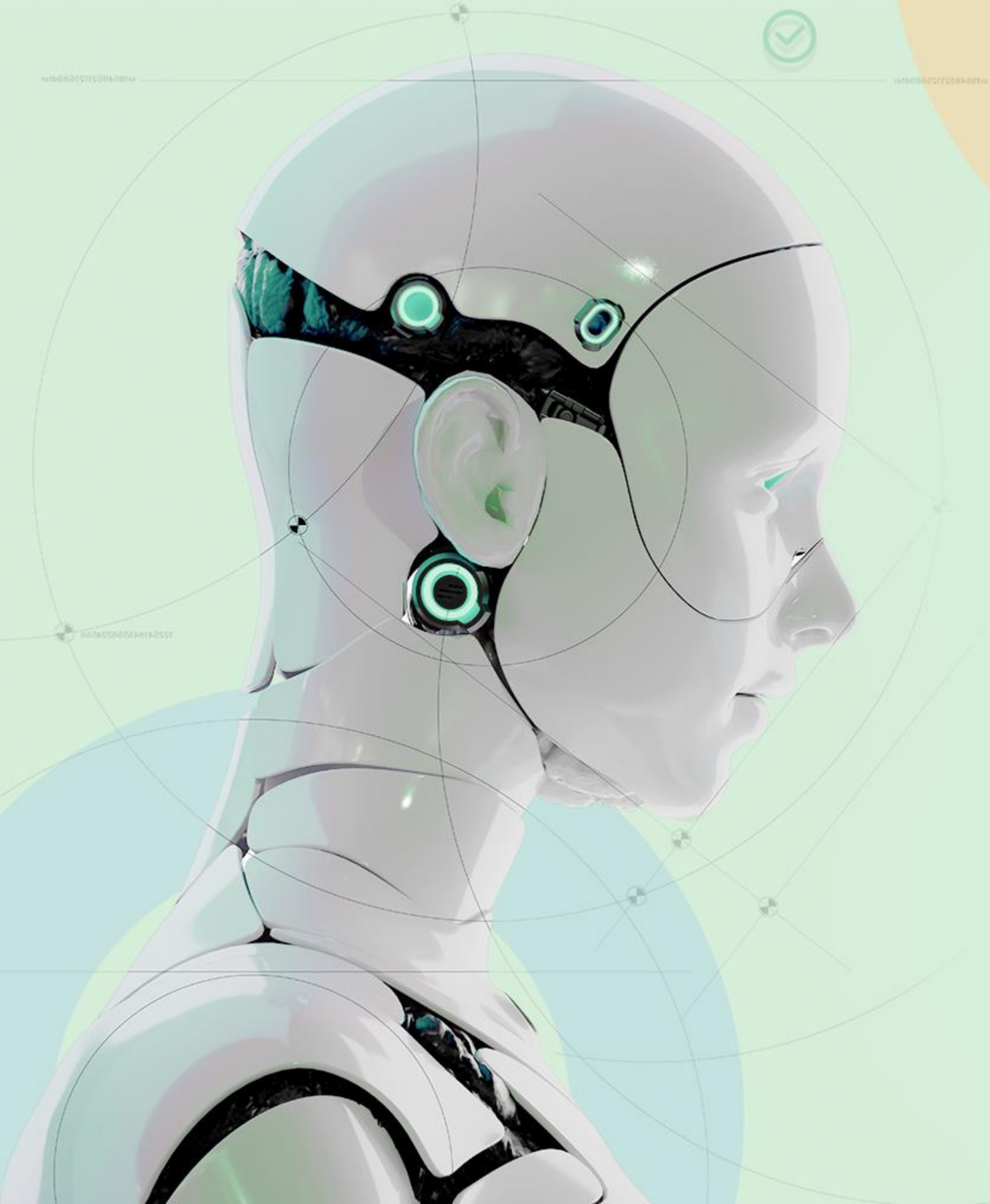
※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

아이디어를 현실로 - **바이브 코딩**

Git과 GitHub 버전 관리

실습

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.



CONTENTS

01 버전 관리의 필요성

02 Git 용어와 GitHub 준비

03 실습

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

지난 시간 복습

실습 & 마무리

📚 지난 차시에서 배운 것

- 5개 핵심 용어 학습
- GitHub 계정 및 레포 생성

🎯 이번 차시 목표

- 실제로 GitHub에 프로젝트 연동
- 커밋 및 푸시 실습
- 핵심 키워드 정리

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

이론은 충분히! 이제 손으로 배워봐요

직접 해볼 시간!

오늘의 실습 체크리스트

- [ ] GitHub 계정 확인 / 가입
- [ ] 새 레포지토리 생성(mylink)
- [ ] Gemini CLI로 Git 연동
- [ ] 첫 커밋 및 푸시
- [ ] GitHub에서 결과 확인
- [ ] 코드 수정 후 재커밋 체험

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

실습 시연 흐름

시연 흐름

1

GitHub 준비

GitHub 로그인
→ 새 레포지토리 생성(mylink)

2

Git 연동

Gemini CLI로 git init
→ add → commit → push

3

확인 + 재커밋

GitHub에서 결과 확인
→ 코드 수정 후 재커밋

각 단계별로 AI와 함께 구현해봐요.

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

이것만 기억하세요!

오늘의 핵심 키워드 3가지

Git

코드의 변경 이력을
자동으로 기록하는 도구

Commit & Push

저장하고 업로드하는
두 가지 핵심 동작

AI 연동

명령어 몰라도
프롬프트로 Git 사용 가능

※ 이 강의는 생성형 AI를 활용하여 제작하였습니다.

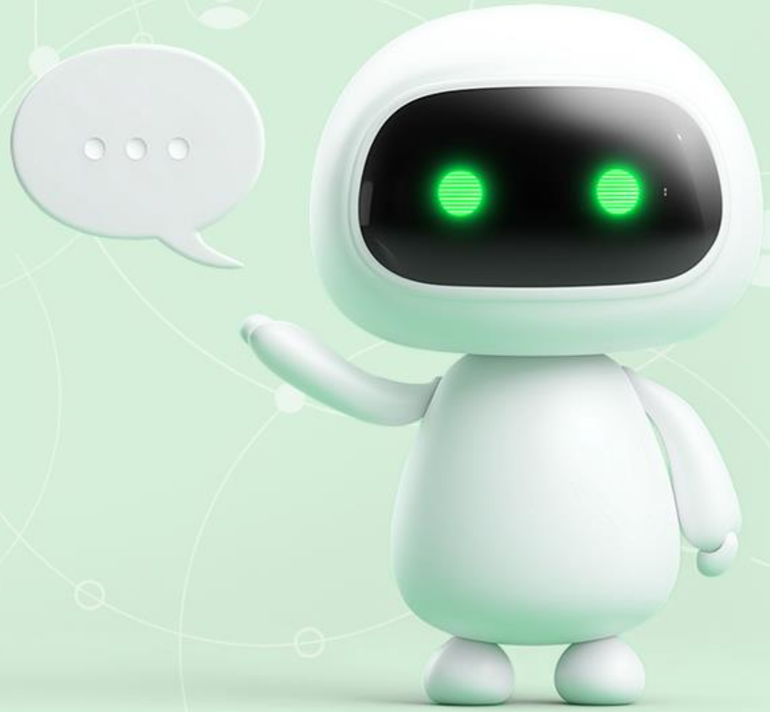
오늘 배운 것

버전 관리의 필요성
"최종_최최종" 문제 해결

Git이란?
코드의 타임머신

핵심 용어 5가지
Repository, Commit, Push, Pull, Clone

AI로 Git 연동
프롬프트만으로 GitHub 업로드



아이디어를 현실로 - **바이브 코딩**

THANK YOU